PROJETO: FinStart



Curso: Ciência de Dados e IA

Disciplina: PI: Programação Web Código: 2181161

Turma: 101

Professor: Fernando Luiz de Almeida Silveira

Nome do Grupo: FinStart

Gabriel Polli Perin – RA: 25007439

Yuri Kikuchi – RA: 25007272

Enzo Cayres Ferreira – RA: 25000144

Eduardo dos Santos Fernandez – RA: 25000693

Paulo Henrique Scavariello Guimarães – RA: 25017738

Campinas, 2º semestre de 2026

1. Ideação do Projeto

3ª entrega: consolidação da implementação. Atualização da publicação em 29/10/2026.

A FinStart é uma plataforma gratuita de educação financeira para jovens que estão começando a organizar renda, despesas e objetivos. Reúne conteúdos educativos, orçamento, projeções de juros compostos e acompanhamento de metas. O problema abordado é a dificuldade de transformar conceitos financeiros em decisões e hábitos de planejamento.

O público inicialmente definido é de jovens de 16 a 30 anos e pessoas com pouco conhecimento sobre finanças. A aplicação não verifica idade nem coleta data de nascimento. A proposta mantém o eixo aprender, simular e planejar, sem movimentar dinheiro ou recomendar produtos financeiros individuais.

A ODS 4 é a contribuição principal, pelo acesso a conhecimento e habilidades de planejamento. A relação com a ODS 8 é indireta: compreender serviços e organizar recursos pode apoiar decisões financeiras. A plataforma não comprova geração de renda, empregos ou impacto econômico mensurável.

Histórico e escopo desta revisão

A primeira proposta concentrava-se no front-end e em armazenamento no navegador. Após o retorno do professor, a arquitetura foi revista para Full Stack. A segunda entrega formalizou HTML/CSS/JavaScript, Node.js/Express, API REST, MySQL e autenticação JWT. Esta revisão apresenta a implementação dessas camadas e as correções posteriores; não atribui os recursos atuais à entrega original de 01/10/2026.

A versão publicada utiliza MySQL persistente na Aiven, API Node.js/Express no Render e interface na Vercel. A instância MySQL local documentada durante o desenvolvimento continua sendo uma alternativa independente. Seus dados pessoais não foram enviados ao banco online. SQLite permanece como alternativa local e de testes; a produção exige MySQL.

2. Protótipo da Solução

O conjunto de telas foi ampliado para representar a jornada do MVP. Início apresenta as ferramentas; Aprender oferece leitura por assunto e nível; Orçamento organiza a renda; Juros e Comparar apresentam cenários; Metas registra objetivos; Meu painel reúne dados da conta; Sobre explica a proposta. O painel aparece no fim do menu, à direita no desktop.

A identidade visual permanece preta e dourada. O menu recolhe em telas pequenas, os campos têm rótulos, o foco de teclado é visível e os gráficos possuem tabelas equivalentes. Estados vazios, avisos de erro e confirmação de operações orientam o uso.

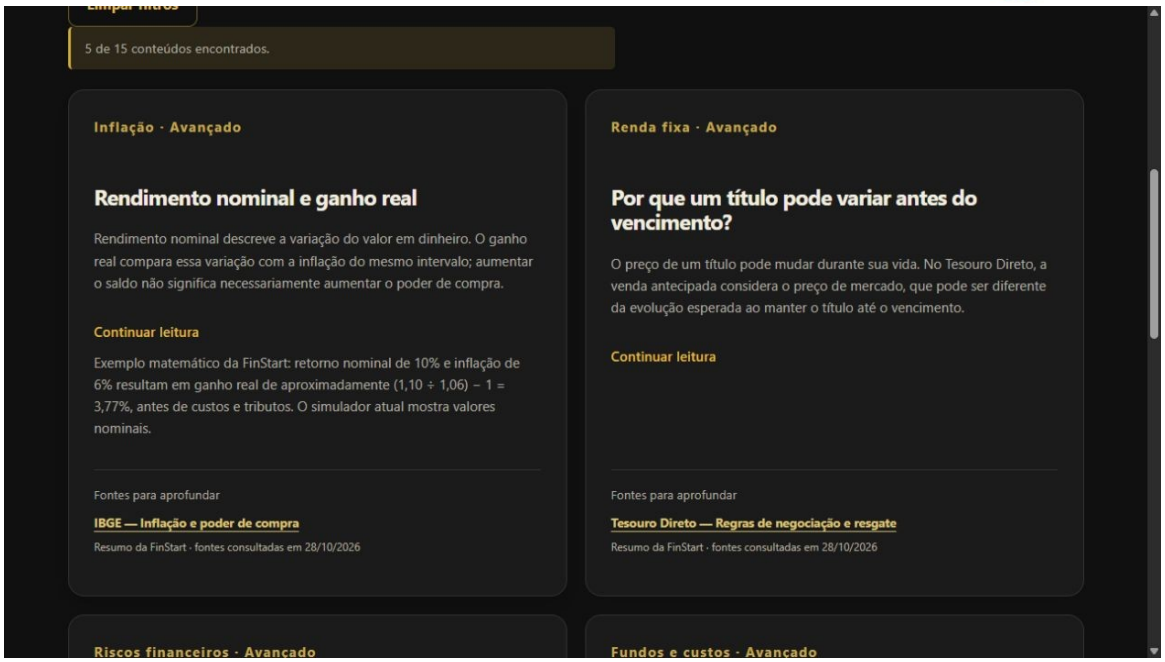


Figura 1 — Biblioteca com filtro avançado, referências oficiais e leitura expansível. Captura da revisão de 28/10/2026.

Fluxos centrais implementados

Visitante: ler conteúdo → simular orçamento ou juros → comparar hipóteses. Conta autenticada: entrar → salvar simulação → criar ou editar meta → registrar saldo → consultar gráfico e painel. Os dados financeiros persistem no servidor; a preferência de filtro pode ficar no navegador.

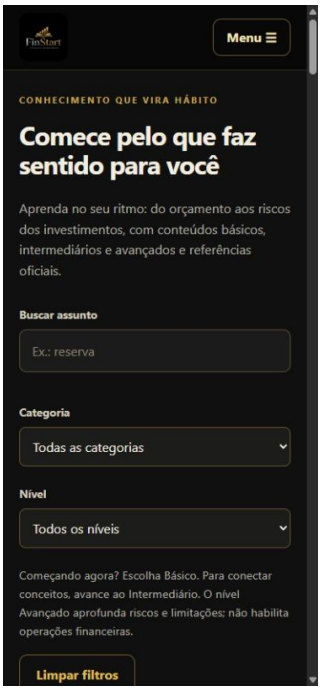


Figura 2 — Biblioteca em viewport de 390 px. Teste em navegador; não representa teste em aparelho físico.

3. Planejamento do Projeto

Prioridade	Item e critério de conclusão
P1	Conta e SQL: cadastro, login, logout e isolamento dos registros verificados.
P1	Ferramentas: orçamento, projeções, comparação e metas com gravação e consulta.
P1	Progresso: prazo, aporte mensal necessário, atualização do saldo e histórico.
P2	Aprendizagem: 15 conteúdos com fontes, busca, categorias e três níveis.
P2	Qualidade: testes de API, responsividade, documentação e preservação de dados.
P2	Entrega: revisão pelo grupo, publicação e apresentação oral.

Kanban e acompanhamento

Concluído: MVP integrado, publicação HTTPS, banco Aiven, testes visuais e mobile online, persistência após reinício da API e backup externo com restauração isolada. A Vercel publica automaticamente a cópia de Yuri no GitHub; o Render usa essa mesma origem, com atualização manual documentada. Pendentes: automação do Render por integração Git, revisão do grupo e envio no Canvas. O quadro anterior em docs/kanban.json registra a revisão local de 28/10; docs/VALIDACAO-FINAL.md registra esta rodada.

O quadro registra o trabalho observável desta revisão. Não há evidência suficiente para reconstruir movimentações, reuniões ou sprints anteriores; por isso, esses eventos não são apresentados como realizados. Para os próximos ciclos, cada item deve ter responsável, critério de conclusão e evidência.

Divisão prevista na segunda entrega

Paulo: autenticação e juros; Eduardo: dados e metas; Enzo: API; Gabriel: progresso e gráficos; Yuri: conteúdos e acompanhamento. Essa distribuição é a prevista no planejamento e precisa ser confirmada pelo grupo para a apresentação; não comprova autoria individual das alterações atuais.

4. Levantamento de Requisitos

ID	Requisito e situação
RF01	Cadastrar, autenticar e encerrar sessão: implementado.

ID	Requisito e situação
RF02	Consultar conteúdos por assunto, categoria e nível: implementado.
RF03	Calcular e salvar orçamento 50/30/20: implementado.
RF04	Projetar juros compostos e mostrar evolução em gráfico/tabela: implementado.
RF05	Comparar cenários e salvar resultados: implementado com taxas hipotéticas.
RF06	Criar, consultar, editar e excluir metas com prazo: implementado.
RF07	Registrar progresso e calcular aporte mensal necessário: implementado.
RF08	Consultar painel e históricos exclusivos da conta: implementado.
RNF01	Separar front-end, API, regras de domínio e persistência SQL.
RNF02	Validar entradas no servidor e restringir operações ao proprietário.
RNF03	Aceitar uso responsivo, teclado e leitura tabular dos gráficos.
RNF04	Manter configuração privada fora do Git, com backup e instruções de execução.

Regras de negócio

O orçamento distribui a renda em 50% necessidades, 30% desejos e 20% futuro como referência educativa. Valores monetários persistem em centavos inteiros. Juros usam taxa mensal constante e aportes ao fim de cada mês. A comparação converte taxas anuais hipotéticas em taxas mensais equivalentes, com os mesmos valores e prazo.

A meta contém valor-alvo, saldo, prazo e data inicial. O saldo não pode ser negativo nem superar o alvo. A atualização e o registro de progresso ocorrem na mesma transação. O aporte necessário divide o valor restante pelos meses restantes, sem supor rendimento. Registros privados exigem sessão válida e pertencimento à conta.

As projeções não descontam inflação, taxas ou tributos. Recuperação de senha, verificação de e-mail, produtos reais e cotações não estão implementados. A hospedagem pública HTTPS já está disponível.

5. Desenvolvimento do Projeto

Arquitetura e organização

Na publicação, o navegador carrega HTML/CSS/JavaScript da Vercel e envia fetch/JSON para /api no mesmo domínio. O proxy encaminha a requisição ao Node.js 24/Express no Render; controllers acessam o MySQL da Aiven por conexão TLS. O navegador não recebe credenciais SQL. O backend mantém organização em camadas; a infraestrutura distribui frontend, aplicação e banco.

server/index.js organiza as rotas; controllers.js executa as operações; auth.js trata autenticação; mysql.js e database.js oferecem persistência; dominio.js e financas.js concentram regras de cálculo. database/ reúne os scripts SQL, tests/ reúne verificações e docs/ contém relatório e evidências. Scripts de inicialização, configuração e backup são utilitários operacionais necessários.

Modelo de dados atualizado

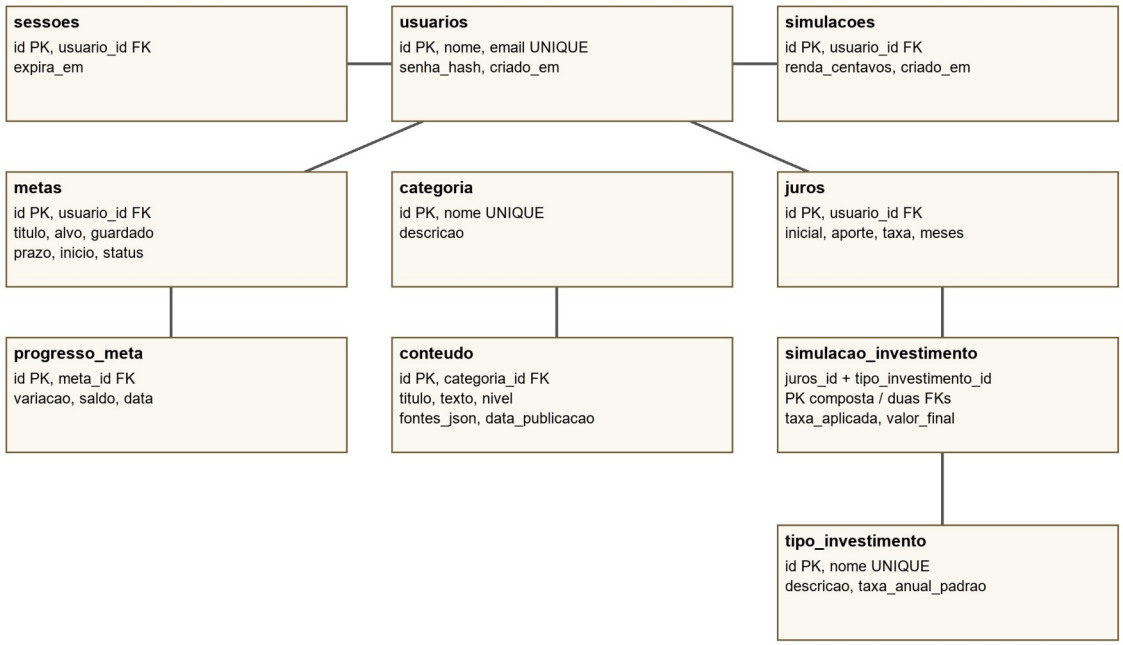


Figura 3 — DER físico simplificado. Cada ligação representa relação 1:N; simulacao_investimento possui chave primária composta e implementa a associação entre projeção e cenário.

Tabela	Função e relacionamento
usuarios	Conta; email único e senha_hash. Pai de sessões, orçamentos, metas e juros.
sessoes	Sessão revogável; usuario_id referencia usuarios.
simulacoes	Orçamentos; usuario_id referencia usuarios.

Tabela	Função e relacionamento
metas	Objetivo, alvo, saldo, prazo, início e status; usuario_id referencia usuarios.
progresso_meta	Variação e saldo por registro; meta_id referencia metas.
juros	Plano de projeção; usuario_id referencia usuarios.
categoria	Classificação dos conteúdos.
conteudo	Texto, nível, referências e data; categoria_id referencia categoria.
tipo_investimento	Cenário didático com taxa anual declarada.
simulacao_investimento	Resultado por juros_id e tipo_investimento_id; duas FKs e PK composta.

O catálogo físico contém 10 tabelas, 8 chaves estrangeiras e 51 colunas. Há chaves primárias, unicidade, checks e exclusão em cascata nas relações de propriedade. O script executável é database/schema.mysql.sql; database/consultas-demonstracao.mysql.sql permite inspecionar tabelas, colunas e relacionamentos sem alterar registros.

Conciliação com o MER anterior

As entidades usuario e meta usam os nomes físicos usuarios e metas. A projeção planejada como simulacao é armazenada em juros, enquanto simulacoes identifica o orçamento. As entidades categoria, conteudo, tipo_investimento, simulacao_investimento e progresso_meta foram implementadas. Sessões e orçamento complementam as oito entidades do planejamento. Novos atributos operacionais incluem prazo, status e referências do conteúdo; nascimento foi omitido por minimização de dados.

Conexão MySQL e migração

Durante o desenvolvimento, foi criada uma instância MySQL exclusiva em 127.0.0.1:3308, preservando o serviço anterior e seus dados. Essa configuração local não é o banco usado pelo endereço público. Na publicação, o backend conecta ao MySQL da Aiven com variáveis privadas e validação do certificado TLS; contas locais e online são independentes.

Na migração local, um backup SQLite foi preservado e os registros foram copiados com IDs, hashes e relacionamentos conferidos. O procedimento está em scripts/configurar-mysql-local.js. Essa migração histórica não transferiu contas para a Aiven.

Rotas e segurança

Rotas	Finalidade
GET /api/conteudos e /api/tipos	Catálogo público e cenários.
POST /api/calcular e /api/comparar	Cálculos validados no servidor.
POST /api/cadastro e /api/login	Criação de conta e autenticação.
GET /api/me; POST /api/logout	Conta atual e revogação da sessão.
GET/POST /api/simulacoes e /api/juros	Orçamentos e projeções por usuário.
GET/POST /api/comparacoes	Comparações persistidas por conta.
GET/POST /api/metas; PUT /api/metas/:id	Criação, consulta e edição de metas.
GET /api/metas/:id/progresso	Histórico protegido de saldo.
DELETE /api/{recurso}/:id	Exclusão autorizada dos recursos suportados.
GET /api/saude	Teste da conexão real com SQL.

Senhas usam scrypt com salt. JWT possui assinatura HS256, expiração e verificação no servidor, associado a sessão revogável. O cookie é HttpOnly e SameSite=Strict, com Secure em produção. Consultas parametrizadas e filtros por proprietário protegem os registros. A API limita corpo e tentativas de autenticação e valida entradas e origem.

Integração demonstrada

Cadastro: formulário → validação e hash → INSERT usuarios. Login: verificação da senha → sessão/JWT → cookie. Meta: formulário autenticado → UPDATE metas e INSERT progresso_meta na mesma transação → consulta do histórico → gráfico/tabela. Comparação: cálculo dos cenários → INSERT juros e resultados associados → consulta no painel.

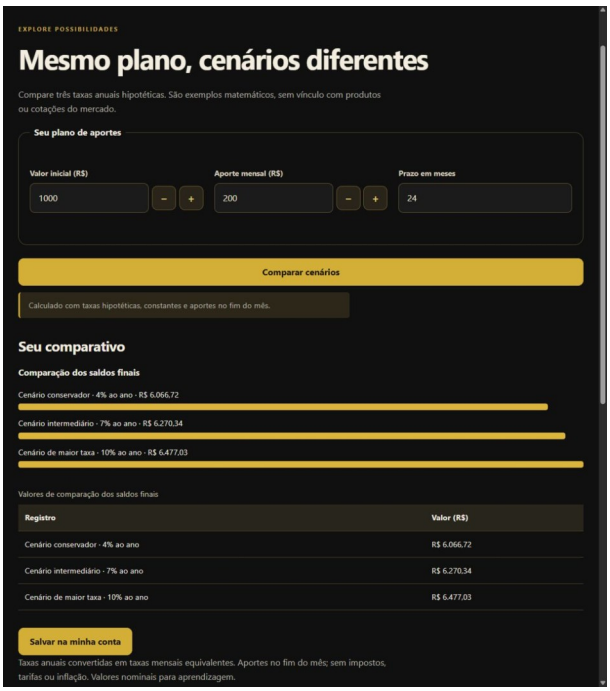


Figura 4 — Comparação salva em teste integrado com MySQL e dados fictícios. As taxas são hipóteses didáticas.

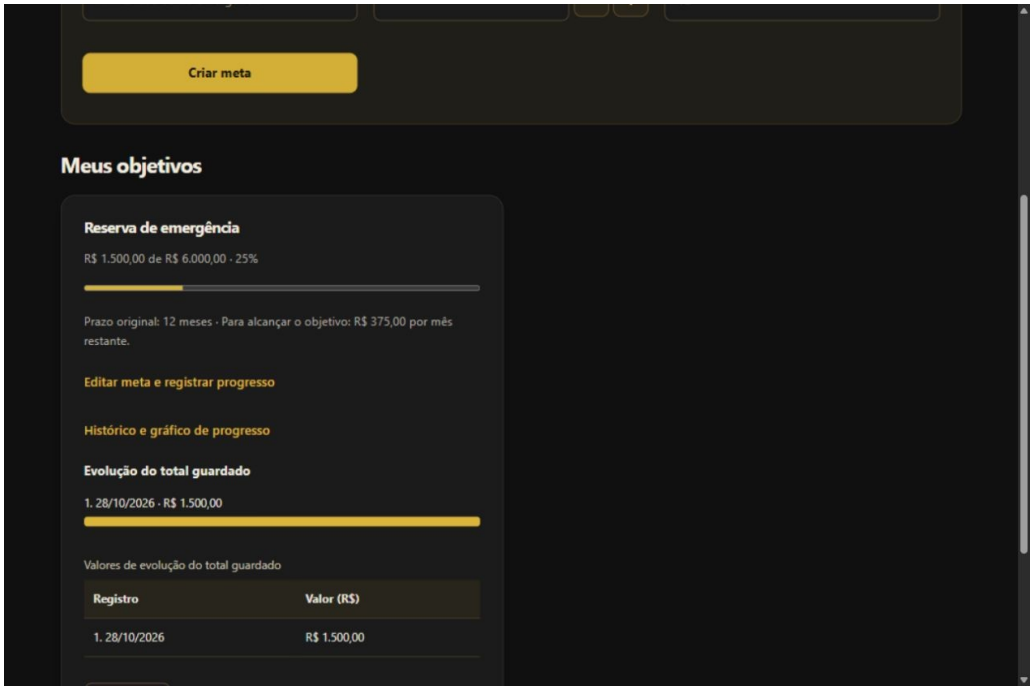


Figura 5 — Meta e progresso consultados pela API em teste integrado com MySQL. Dados fictícios de demonstração.

Aplicação das boas práticas das aulas

Foram confrontados os 15 PDFs fornecidos, totalizando 312 páginas, com o código e os critérios da atividade. Os princípios aplicáveis foram incorporados sem tratar exemplos didáticos como requisitos obrigatórios de infraestrutura.

Aulas e foco	Aplicação verificada / decisão
01 — problema, público e ODS (22 p.)	Público de 16–30 anos, educação gratuita, ODS 4 principal e contribuição indireta à ODS 8; sem promessa de renda.
02 — ciclo, Scrum e Kanban (32 p.)	Backlog por prioridade, definição de concluído, quadro e registro datado da revisão. Reuniões anteriores não comprovadas não são declaradas.
03 — UI/UX (17 p.)	Jornada completa, identidade preto/dourado, estados vazios, feedback, prevenção de exclusão acidental e navegação consistente.
04 — arquitetura (25 p.)	Monólito em camadas, Express/API/controllers/adaptadores SQL; decisões e diferenças documentadas.
05 — Git e organização (18 p.)	Código formatado, README operacional, dependências fixadas, diretórios de servidor/SQL/testes/docs e .gitignore de dados privados.
06 — HTML e SEO (28 p.)	HTML5, idioma, UTF-8, viewport, títulos e descrições; conta marcada noindex; landmarks e texto alternativo.
07 — semântica textual (17 p.)	Hierarquia h1/h2/h3, parágrafos e conteúdos por nível com referências oficiais.
08 — listas e tabelas (22 p.)	Menu em lista; tabela de dados junto dos gráficos, caption, thead/tbody e cabeçalhos scope. Não se usa tabela para layout do site.
09 — formulários (15 p.)	Labels, required, tipos, limites, agrupamento em fieldset no comparativo; credenciais em POST; validação no servidor.
10 — CSS (19 p.)	Estilo externo, variáveis, fonte de sistema, foco visível e contraste; remoção do CSS inline do orçamento.
11 — layout e responsividade (23 p.)	Grid/Flex, controles de toque, menu mobile, tabelas com área de rolagem. Novos componentes com base mobile e expansão por min-width.
12 — JavaScript (17 p.)	Módulos, const/let, funções reutilizáveis, conversão numérica explícita, regras de domínio e limites verificados.
13 — DOM e eventos (25 p.)	addEventListener, preventDefault, input/change, DOM com textContent, aria-invalid e avisos de operação.
14 — Storage (11 p.)	sessionStorage somente para preferências de categoria e nível; contas, metas e simulações no SQL.
15 — JWT (21 p.)	JWT real assinado/expirado/verificado no servidor e sessão revogável. Cookie HttpOnly; o exemplo didático de token simulado em Storage não é usado como segurança.

Biblioteca e referências

A biblioteca reúne 15 resumos: cinco básicos, cinco intermediários e cinco avançados, distribuídos por 13 categorias. Inclui orçamento, reserva, inflação, crédito, diversificação, metas, segurança, renda fixa, custos e riscos. Cada artigo identifica as fontes oficiais e a data de consulta; busca textual e filtros podem ser combinados.

6. Testes

A revisão local de 28/10 registrou dez testes aprovados e integração MySQL separada. Na preparação de produção, a suíte registrou 11 testes aprovados e um teste MySQL opcional ignorado. Em 29/10, scripts/verificar-online.mjs foi executado contra o domínio público da Vercel, exercitando proxy, API e MySQL reais com contas fictícias. Os resultados abaixo distinguem a verificação local das evidências online; não constituem auditoria completa de segurança.

Verificação	Resultado e alcance
Autenticação e autorização	Cadastro/login/logout, JWT e isolamento entre contas aprovados.
CRUD e persistência	Orçamento, juros, metas e comparações; leitura após reabertura aprovada.
Transações MySQL	Rollback e relacionamentos/cascatas comprovados no teste isolado.
Cálculos e validação	Aportes ao fim do mês, taxa zero, limites e entradas inválidas verificados.
HTTP e formulários	JSON inválido, tamanho, origem e UTF-8; envio de credenciais protegido.
Migração local	Todos os campos originais comparados antes do commit; SQLite preservado.
Responsividade	Oito páginas online em 320, 390, 768 e 1280 px; menu, tabelas e fluxos mobile verificados nesta rodada. Sem teste em aparelho físico.
Aprendizagem	15 conteúdos, cinco por nível, fontes HTTPS, busca, filtros e estado vazio verificados.

Casos para reprodução

Juros: R$ 1.000 iniciais, R$ 200 ao fim de cada mês, taxa de 0,5% mensal e 24 meses resultam em R$ 6.213,55; o valor investido é R$ 5.800 e os juros, R$ 413,55. Meta: alvo de R$ 6.000 em 12 meses e saldo inicial de R$ 1.500 indicam 25% e aporte de R$ 375, enquanto restarem 12 meses.

Consultar docs/evidencias/testes.json, mysql.json, endpoints.json, banco.json e biblioteca-breakpoints.json. As evidências SQLite documentam os testes dessa alternativa; mysql.json documenta o esquema MySQL. As capturas com dados fictícios demonstram o sistema em execução. Não foram realizados testes em aparelho físico ou uma auditoria completa com leitores de tela.

Verificação da publicação em 29/10/2026

O script scripts/verificar-online.mjs foi executado novamente pelo domínio público após as atualizações. Passaram: saúde SQL, cadastro, login, cookie Secure/HttpOnly/SameSite=Strict, cálculos, gravação e consulta das ferramentas, progresso, comparação, isolamento entre contas, logout, recuperação em novo login e exclusão dos registros fictícios. Contas de teste permanecem; não se presume que existam somente duas. Dados inseridos posteriormente foram preservados.

Na interface publicada, foram verificados cadastro, login, orçamento, projeção, meta, edição e histórico, comparação, biblioteca e exclusões. Playwright com Microsoft Edge verificou as oito páginas em 320, 390, 768 e 1280 px: sem transbordamento horizontal e sem erros JavaScript na rodada final. Menu mobile, gráficos e tabelas foram exercitados. O teste não representa uso em aparelho físico. Após reinício controlado da API às 23h23, a meta de R$ 6.000 com R$ 1.500 guardados foi recuperada em nova aba e recarga. Caminhos privados retornaram 404. Evidências: docs/evidencias/validacao-final/.

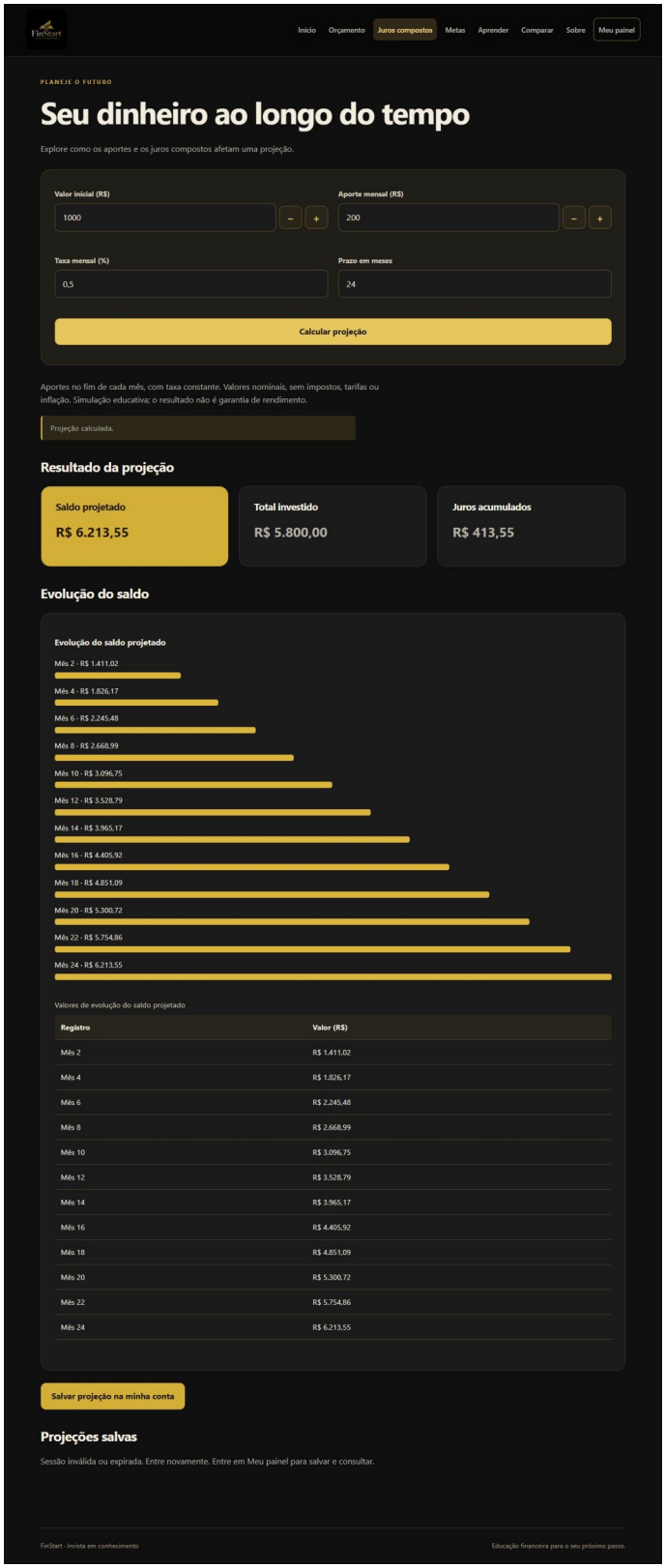


Figura 6 - Projeção calculada no site público da Vercel em 29/10/2026. Evidência: docs/juros-online.png.

7. Implantação

A apresentação utiliza https://finstart-publicacao.vercel.app. Basta internet e navegador; não é necessário instalar Node.js, MySQL ou abrir terminal no computador do professor. O frontend está na Vercel, a API em https://finstart-api-l60v.onrender.com e o banco persistente na Aiven. GET /api/saude pelo domínio do site confirmou banco conectado em 29/10/2026.

A Vercel publica somente os arquivos selecionados por scripts/build-static.js. vercel.json define o proxy /api para o Render e cabeçalhos de segurança. O Render executa pnpm install --frozen-lockfile e node server/index.js. server/index.js usa PORT e HOST=0.0.0.0 em produção; PUBLIC_ORIGIN limita a origem aceita. A conexão TLS está em server/mysql.js e o esquema em database/schema.mysql.sql.

No Render, NODE_ENV=production, DB_DRIVER=mysql e DB_SSL=true identificam a configuração. DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, DB_SSL_CA e JWT_SECRET ficam nas variáveis privadas. Nenhuma senha deve ser mostrada na apresentação ou incluída no ZIP. O Workbench é opcional para inspeção autorizada do banco; abrir um arquivo SQL sozinho apenas mostra o código.

Backup e compartilhamento

GitHub e ZIP preservam código e documentos; não guardam os dados online. O painel Aiven confirmou Running no plano Free e mostrou um backup completo do provedor. Um dump externo com TLS VERIFY_IDENTITY foi restaurado em MySQL 8.0.45 local isolado; as dez tabelas coincidiram por contagem e hash. A instância temporária foi encerrada. Não houve restauração sobre a produção nem em outra Aiven 8.4. O dump privado fica separado do pacote público. Frequência, retenção externa e responsável pelos próximos backups estão documentados como manutenção.

A pedido de Yuri, foi criada https://github.com/yurikikuchi2006/finstart, preservando o histórico da origem enzocayresferreira7-pixel/finstart. O projeto existente da Vercel foi conectado à cópia; novos commits dispararam deploys Ready no mesmo endereço público. O build seleciona 25 arquivos públicos para dist. O serviço Render existente também usa a cópia de Yuri, mantendo Node nativo, variáveis privadas e banco. Pela conexão de repositório público utilizada, seus próximos deploys são manuais; não se declara automação da API comprovada. O frontend mantém espera de 90 segundos, sem repetição de gravações.

8. Manutenção / Evoluções Futuras

Evolução em relação à segunda entrega

O protótipo anterior concentrava formulário inicial, página institucional e orçamento. A versão atual acrescenta fluxos completos de cadastro/login, biblioteca, juros,

comparação, metas, progresso e gráficos. A API, autenticação e persistência passaram de planejamento para implementação integrada, mantendo a arquitetura Node.js/Express/MySQL definida na segunda entrega.

Problema observado	Correção aplicada
Telas do MVP incompletas	Oito páginas e fluxos de leitura, simulação e acompanhamento.
Banco/API/JWT apenas planejados	Controllers, consultas reais, JWT verificado e MySQL operacional.
HTML local sem executar módulos/API	Inicializador HTTP e bloqueio orientado de file://.
Senha aparecendo na URL	Formulários POST explícitos e proteção antes dos módulos.
Saldo sem histórico e prazo	Registro transacional, prazo e cálculo mensal necessário.
Conteúdo apenas introdutório	Biblioteca com três níveis e fontes oficiais.
Kanban sem trabalho em andamento	Quadro atual datado, com critérios de conclusão e pendências reais.

Próximos passos: integrar o provedor Git ao Render para atualização automática; estabelecer retenção e rotina protegida de backups, repetindo a restauração em ambiente isolado; recuperação de senha e verificação de e-mail; testes com usuários e acessibilidade assistiva; monitoramento e revisão das fontes. Foram corrigidos os retornos de caminhos inexistentes por rotas 404 explícitas e o identificador de conteúdo da página Sobre, que causava erro JavaScript e prejudicava o link de salto. Os cenários financeiros continuam hipotéticos.

9. Considerações Finais

A FinStart mantém a proposta educativa original e apresenta integração entre interface, API e banco relacional. A consolidação está demonstrada pelos fluxos implementados, scripts SQL, modelo atualizado e verificações registradas. O grupo deve revisar o pacote e conseguir explicar os cálculos, as relações entre tabelas e os limites da versão durante a apresentação.

Tópico exigido na 3ª entrega	Localização neste documento
1 Front-end implementado	Seções 2, 4 e 5; capturas das telas.
2 Back-end funcionando	Seções 5 e 6; rotas, controllers e testes.

Tópico exigido na 3ª entrega	Localização neste documento
3 Banco consolidado	Seção 5; DER, tabelas, FKs, SQL e conexão.
4 Integração real	Seções 5 e 6; fluxos completos e persistência.
5 Evolução desde a 2ª entrega	Seção 8; alterações, desafios e próximos passos.
6 Clareza e organização	Modelo oficial, estrutura cumulativa e evidências identificadas.

Referências documentais e técnicas

Documentos do grupo: primeira entrega original e corrigida; 2a-Entrega-FinStart; Relatorio_Parcial_1_FinStart. Materiais da disciplina: aulas 01 a 15; comentários de Fernando Luiz de Almeida Silveira; enunciado da terceira entrega; documento padrão oficial fornecido pelo professor.

Documentação técnica: Node.js (https://nodejs.org/docs/latest-v24.x/api/), Express (https://expressjs.com/), MySQL (https://dev.mysql.com/doc/refman/8.0/en/) e SQLite (https://www.sqlite.org/).

Fontes educativas: Banco Central, CVM/Portal do Investidor, IBGE e Tesouro Direto. Os endereços específicos, títulos e data de consulta estão em cada conteúdo da biblioteca e no catálogo versionado server/conteudos.js. Consulta dos conteúdos: 28/10/2026.

Pendências acadêmicas: revisão do grupo, upload no Canvas e apresentação oral. Código e aplicação estão publicados. As verificações desta rodada estão em docs/VALIDACAO-FINAL.md, com evidências separadas dos dados privados. Permanecem a automação do Render e as melhorias descritas na seção 8. Testes finitos não garantem ausência de falhas, disponibilidade futura dos provedores ou substituição da avaliação do professor.